智能笔记 App（Android）需求与技术方案

- 文档版本：v0.1
- 日期：2026-03-04
- 目标平台：Android 手机 / 平板

---

1. 背景与目标

开发一款面向学习与知识整理的 Android 笔记应用，支持 **PDF/拍照输入**，自动识别文本、图片、表格等并结构化为 **Markdown**（或内部结构），结合用户的 **本地知识库（教材/PPT/讲义）** 进行 AI 补充、纠错、问答，最终通过 **Typst** 自动排版输出高质量 PDF。

核心目标

1. 多模态输入（PDF / 拍照）→ 可编辑的结构化笔记（Markdown）
2. 知识库加持（RAG）→ 可引用来源的补充/纠错/问答
3. 一键排版（Typst）→ 输出观感稳定、可定制的 PDF

---

2. 用户与主要场景

目标用户

- 学生：课堂笔记、板书、PPT拍照、教材整理
- 自学用户：导入课程资料→自动生成讲义式笔记→提问巩固
- 复习用户：旧笔记统一纠错与排版，导出可打印 PDF

关键场景

- S1：导入一份课堂笔记 PDF → 自动转 Markdown → 编辑 → 导出 Typst PDF
- S2：拍摄板书/PPT → OCR + 结构化 → AI 补步骤/解释 → 自动排版
- S3：导入教材/PPT 成知识库 → 笔记补充/答疑时引用知识库原文位置
- S4：用户修改前文定义/符号 → 后续相关补充内容提示“可能失效”并一键重算

---

3. 功能需求（含验收要点）

3.1 输入与导入（PDF / 拍照）

需求

- 支持导入 PDF（单个/批量）
- 支持拍照上传（单张/多张）与相册导入
- 图片预处理：自动裁切、透视矫正、去阴影/去噪、自动旋转

验收要点

- 拍照后 1 次点击可完成“矫正后预览”
- PDF 可按页浏览，支持选择页范围进行识别

- 导入过程可后台执行并展示进度，可取消

3.2 识别与结构化（OCR + 版面 + 表格/图片）

需求

- 自动识别并提取：文本、标题层级、段落、列表、图片、表格
- 扫描 PDF/图片走 OCR；含文本层 PDF 优先抽取文本层（更准更快）
- 输出为 Markdown（建议内部先落地为 Document AST，再导出 Markdown）

验收要点

- 同一页内容顺序与层级基本正确（标题不混入正文）
- 图片能被裁切成资源文件并在笔记中引用
- 表格可输出为 Markdown table（MVP 可允许复杂表格降级为图片+可选手动转表）

3.3 笔记编辑与管理

需求

- 笔记列表：按课程/标签/时间排序
- 笔记编辑：Markdown 编辑 + 预览（或结构化编辑器 + 导出 Markdown）
- 资源管理：图片/原 PDF 页关联保存，可回看原图定位

验收要点

- 编辑后自动保存（可设置）
- 可一键跳回“原图/原 PDF 页”定位核对
- 支持导出 Markdown（含资源相对路径）

3.4 Markdown → Typst 自动排版与导出 PDF

需求

- 将笔记（Markdown 或 AST）转换为 Typst 文档
- 通过 Typst 编译生成 PDF
- 支持模板（主题）选择：简洁/讲义/论文风（MVP 至少 1 套 + 可扩展）

验收要点

- 导出 PDF 版式稳定（标题层级清晰、分页合理、图片不溢出）
- 模板切换不改变内容，仅改变样式
- 导出产物：`.pdf`（可选同时导出 `.typ`）

3.5 本地知识库（课本 / PPT / 讲义）

需求

- 用户可导入资料形成“课程知识库”
- 资料切分（按页/段落/标题）→ Embedding → 向量索引
- 检索用于：补充、纠错、问答，且生成内容必须带引用

验收要点

- 导入后能检索到相关片段（语义检索 + 关键词检索至少其一可用）
- 引用格式包含：来源文件名 + 页码/段落定位（最少做到页码）
- 知识库更新支持增量（追加文档后不需全量重建）

3.6 AI 笔记补充（板书/PPT/笔记 PDF）

需求

- 对识别后的笔记进行补充：缺失步骤、概念解释、推导说明、例题、易错点
- 支持“补充强度”：轻度/中度/重度
- 生成结果作为独立块插入，保留可追溯来源（knowledge citations + input anchor）

验收要点

- 生成内容不覆盖用户原文；以“补充块/注释块”方式插入
- 每个补充块至少包含 1 条引用（若知识库为空则标记“无引用/基于常识”）
- 用户可一键撤销补充块

3.7 智能纠错

需求

- OCR 错字纠正（基于上下文+领域词典/知识库）
- 术语一致性（同义写法统一）
- 可选语言润色（不改变专业含义）
- 以“建议（diff）”呈现，用户确认后应用

验收要点

- 不自动强制改写整段；默认给出修改建议
- 可对单个建议逐条应用/忽略
- 被应用的修改可撤销

3.8 笔记内提问（Q&A）并排版

需求

- 用户在笔记中选中文本或指定范围提问
- 系统结合笔记上下文 + 知识库检索回答
- 回答以 Q&A 模块插入，参与 Typst 排版

验收要点

- 回答包含引用（同 3.5）
- 支持“插入到当前位置”或“插到文末 Q&A 区”
- 导出 PDF 后 Q&A 格式清晰

3.9 编辑后联动更新（依赖追踪）

需求

- 记录 AI 生成块与其依赖（笔记片段锚点、引用片段 id）
- 用户编辑前文后：相关生成块标记“可能过期”，支持一键重算（半自动）

验收要点

- 修改后不会悄悄改写后文；只做提示与建议更新
- 一键重算只更新受影响块，保留用户手写编辑内容
- 提供“查看变更(diff)”再应用

---

4. 数据与核心概念（建议）

4.1 核心对象

- **Note**：一篇笔记（属于某课程/项目）
- **Asset**：图片/原 PDF 页截图等资源文件
- **Document AST**（内部结构，推荐作为主存储）
- **KnowledgeBase**：某课程下的资料集合
- **Chunk**：知识库切分片段（带来源定位）
- **Citation**：引用（source_doc + page + chunk_id + text_span）
- **GenBlock**：AI 生成块（补充/纠错建议/Q&A），含 provenance

4.2 Document AST（简化建议）

- Heading(level, text)
- Paragraph(runs…)
- List(items…)
- Table(rows…)
- Image(asset_id, caption?)
- Callout(type:补充/纠错/Q&A/提示, content…)

- MathInline / MathBlock（可选，MVP 可先不做识别）

> 说明：AST 存储可避免直接操作 Markdown 导致的“改一处全乱”，也利于联动更新与排版。

---

5. 系统架构（推荐：端 + 服务器）

5.1 架构概览

- Android 端：负责导入、预处理、编辑、展示、任务调度、离线缓存
- Python 服务端：负责 OCR/版面分析、RAG 检索、LLM 推理、Typst 编译
- 存储：PostgreSQL + pgvector；资源文件存对象存储（或本地磁盘）

5.2 服务拆分（逻辑）

- **ingest-service**：文件上传、任务编排、元数据
- **parse-service**：PDF/图片解析、OCR、表格/图片切块、生成 AST
- **rag-service**：知识库切分、embedding、检索、引用生成
- **ai-service**：补充/纠错/Q&A（输出 AST patch 或块级结构）
- **typst-service**：AST/Markdown → Typst → PDF（调用 Typst CLI）

---

6. 关键流程（端到端）

6.1 拍照 → 笔记

1. 端上拍照 → 预处理（矫正/去阴影）
2. 上传图片（或批量）→ 触发 parse 任务
3. OCR + 版面分析 → AST
4. AST → Markdown（展示/编辑）
5. 用户编辑保存

6.2 导入 PDF → 笔记

1. 选择 PDF → 选择页范围（可选）
2. 若有文本层：抽取文本 + 坐标；否则 OCR
3. 生成 AST → Markdown
4. 可跳回原页核对

6.3 知识库导入 → 可检索

1. 导入资料（PDF/PPT转PDF）
2. 切分 chunks（保留页码/标题）
3. embedding → pgvector
4. 检索可用

6.4 AI 补充 / 问答

1. 生成检索查询（基于当前笔记片段）
2. RAG 检索 top-k chunks
3. LLM 生成（输出“补充块/回答块 + citations + anchor”）
4. 插入笔记并可导出排版

6.5 导出 PDF

1. AST/Markdown → Typst（套模板）
2. Typst compile → PDF
3. 回传/下载/分享

---

7. API（草案，供前后端对齐）

- `POST /v1/upload`：上传 PDF/图片，返回 file_id
- `POST /v1/parse`：触发解析任务（file_id, pages, mode）
- `GET  /v1/tasks/{task_id}`：任务进度/状态
- `GET  /v1/notes/{note_id}`：获取笔记（AST + Markdown 预览）
- `PATCH /v1/notes/{note_id}`：保存用户编辑（建议以块级/AST patch）
- `POST /v1/kb/import`：导入知识库资料
- `POST /v1/augment`：AI 补充（note_id, range, strength）
- `POST /v1/proofread`：纠错建议（note_id, range）
- `POST /v1/ask`：笔记内问答（note_id, question, range）
- `POST /v1/export/pdf`：导出 PDF（note_id, template_id）

---

8. 技术栈与语言（推荐）

8.1 Android（客户端）

- 语言：**Kotlin**
- UI：Jetpack Compose
- 相机：CameraX
- 后台任务：WorkManager
- 本地存储：Room + 文件系统（assets）
- PDF 渲染：PdfRenderer（起步）/ PDFium（更强）

8.2 服务端（推荐采用服务器）

- 语言：**Python**
- Web 框架：FastAPI
- 任务队列：Celery + Redis（或 RQ）

- 数据库：PostgreSQL + pgvector
- 文档解析：PyMuPDF
- 图像处理：OpenCV
- OCR：PaddleOCR（中英效果好，服务端更合适）
- RAG 框架：LlamaIndex 或 LangChain（任选其一，保持统一）
- Typst：Typst CLI（Rust 实现，但无需写 Rust；Python 调用即可）

---

9. MVP 里程碑（建议）

MVP-1（可用的“识别→编辑→导出”闭环）

- PDF/拍照导入 + 预处理
- OCR/抽取 → AST/Markdown
- Markdown 编辑与资源回看
- Typst 模板 1 套 + 导出 PDF（服务端编译）

MVP-2（差异化：知识库 + 补充 + 纠错）

- 知识库导入与检索引用
- AI 补充（强度可选）
- 纠错建议（diff 模式）
- 笔记内问答（带引用）

MVP-3（高级体验）

- 联动更新（过期提示 + 一键重算 + diff）
- 模板系统扩展（多主题 + 自定义参数）
- 多设备同步（可选）

---

10. 风险与对策（简要）

1. **表格/复杂版面还原难**
    - 对策：MVP 复杂表格先降级为图片块；后续引入更强表格模型/规则重建
2. **端上全做会耗电/慢**
    - 对策：OCR/LLM/Typst 编译放服务端；端上只做预处理与交互
3. **直接存 Markdown 不利于联动更新**
    - 对策：内部以 AST 为主存储；Markdown/Typst 为导出视图
4. **AI 生成内容可能“胡编”**
    - 对策：强制 RAG 引用；无引用时显式标注；生成块独立且可撤销

---

11. 约定（实现建议）

- AI 输出统一为结构化结果（AST block 或 JSON patch），避免直接让模型输出大段 Markdown

- 每个生成块必须记录 provenance：
  - input_anchor（对应笔记范围）
  - citations（知识库来源）
  - model_config（便于复现与调参）
- 默认“建议模式”优先于“自动覆盖”，保证可控与可信